

Lập trình xây dựng Gameplay



CHƯƠNG 3: LẬP TRÌNH XÂY DỰNG GAMEPLAY CỐT LÕI



3.1. ĐỐI TƯỢNG VÀ MÀN CHƠI

- Quản lý GameObject & Component
- Tối ưu & Tái sử dụng đối tượng với Prefabs
- Phân chia và nạp Scene



3.2. ĐIỀU KHIỂN NHÂN VẬT VÀ CAMERA

- Xử lý Input (Input System)
- Chuyển động nhân vật (Character Movement)
- Thuật toán Camera Follow (Offset / Smooth Damping)



3.3. TƯƠNG TÁC VÀ HỆ THỐNG VẬT LÝ (2D/3D PHYSICS)

- Rigidbody & Tính toán lực (AddForce, Velocity)
- Colliders & Physics Material (Ma sát, Độ nảy)
- Xử lý va chạm thực (Collision: OnCollisionEnter)
- Xử lý va chạm vùng (Trigger/Pickups: OnTriggerEnter & Tags)



3.4. TRÌNH BÀY GIAO DIỆN NGƯỜI DÙNG (UI/HUD)

- Canvas System & RectTransform
- TextMeshPro (TMP) & Hiển thị thông số (Thanh máu, Điểm số)
- Xây dựng Menu Game (Main Menu, Pause, Win/Loss)
- Xử lý điều kiện Thắng/Thua (Win/Loss Condition)



3.5. NGHE - NHÌN VÀ HIỆU ỨNG

- Xử lý Animation (Sprite Animation / Animator Controller cơ bản)
- Tích hợp Âm thanh (AudioSource, AudioListener, SFX, BGM)
- Hiệu ứng hạt cơ bản (Particle System cho VFX)



3.6. XỬ LÝ TRẠNG THÁI VÀ TIẾN TRÌNH

- Game State Manager (Quản lý trạng thái: Menu, Play, Pause, GameOver)
- Vòng lặp Coroutine & Timer (Đếm ngược thời gian, Delay logic)
- Lưu trữ & Phục hồi dữ liệu (PlayerPrefs, JSON, ScriptableObject)



Học theo lộ trình logic, dễ hiểu



Tập trung xây dựng Gameplay cốt lõi



Áp dụng thực hành qua Project & Mini Game



Nền tảng vững chắc cho các chương phát triển nâng cao

Đối tượng và màn chơi

GameObject & Component

GameObject: Là thực thể/vật chứa rỗng đại diện cho các đối tượng trong thế giới game.

Component Architecture: Gắn các tính năng cụ thể để định hình hành vi đối tượng:

- Transform: Quản lý vị trí, góc xoay, tỷ lệ.
- Mesh / SpriteRenderer: Hiển thị hình ảnh 2D/3D.
- Rigidbody & Collider: Tính toán vật lý & va chạm.
- Script (MonoBehaviour): Lập trình logic bằng C#.

Prefab & Scene Management

Prefab System:

- Đóng gói đối tượng thành mẫu chuẩn tái sử dụng (Coins, Enemies, Bullets).
- Đồng bộ chỉnh sửa trên tất cả phiên bản (Instances) xuất hiện trong màn chơi.

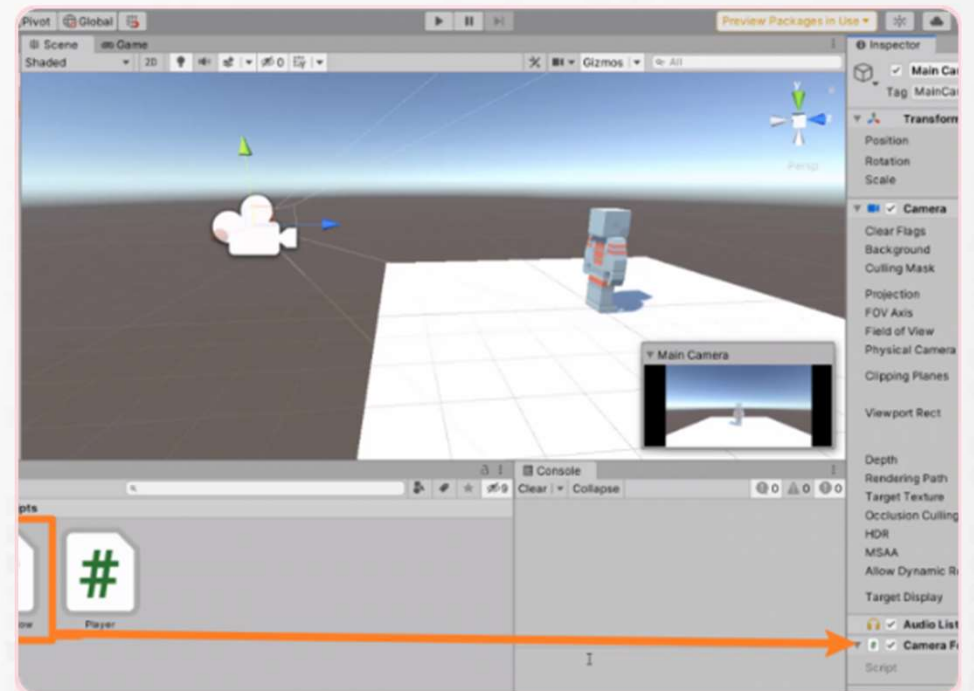
Scene & Level Organization:

- Phân chia các màn chơi, Menu, hoặc không gian môi trường.
- Quản lý nạp Scene linh hoạt (Additive Loading / Asynchronous Loading).

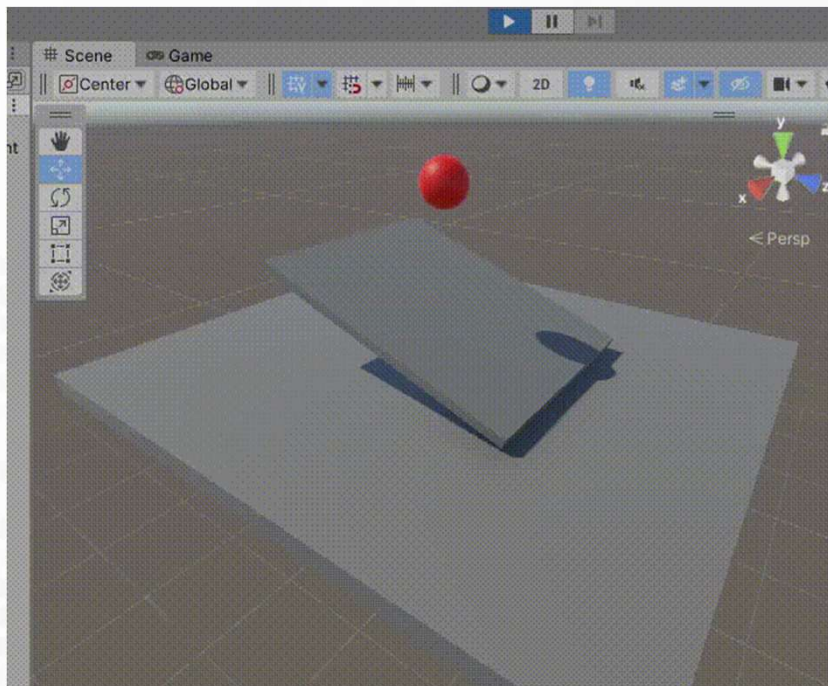
Input System & Camera Follow

- ❗ **Xử lý Input linh hoạt:** Đọc tín hiệu điều khiển từ bàn phím, chuột hoặc tay cầm thông qua Input System (Old vs New Input System).
- 🚶 **Di chuyển nhân vật:** Áp dụng vận tốc `rb.velocity` hoặc tác động lực `AddForce()` dựa trên trục điều khiển `Input.GetAxisRaw()`.
- 📹 **Camera Follow Script:** Tính toán khoảng cách bù (Offset) giữa Camera và Player để góc nhìn di chuyển mượt mà:

```
transform.position = player.position + offset;
```



Hệ thống Vật lý & Tương tác



Hệ thống Vật lý & Tương tác

Xử lý va chạm

Tiêu chí	Collision (Va chạm vật lý)	Trigger (Va chạm nhận diện)
Cấu hình	Rigidbody + Collider (Is Trigger = false)	Rigidbody + Collider (Is Trigger = true)
Phản ứng Vật lý	Có lực cản, nảy sinh dội ngược, không cho xuyên qua nhau.	Cho phép đối tượng đi xuyên qua mà không cản trở.
Hàm Sự kiện (C#)	OnCollisionEnter() / OnCollisionExit()	OnTriggerEnter() / OnTriggerExit()
Ứng dụng thực tế	Tường bao, mặt đất, vật cản cứng, va chạm giữa đạn và quái.	Vùng ăn Coin/vật phẩm, cổng chuyển cảnh, điểm lưu Checkpoint.

Giao diện người dùng (UI/HUD)



Canvas & HUD

UI Toolkit / Canvas: Quản lý không gian vẽ giao diện (Screen Space / World Space).

HUD: Hiển thị thanh máu (HP Bar), điểm số (Score), thời gian bằng TextMeshPro.



Hoạt ảnh, Âm thanh và Hiệu ứng hạt (Animation, Audio và Particle System)



Animation

Animator Controller: Quản lý FSM (Finite State Machine) chuyển đổi trạng thái hoạt hình (Idle, Run, Jump).

Sprite / Skeleton: Cắt chuỗi khung hình 2D hoặc sử dụng bộ xương 3D.

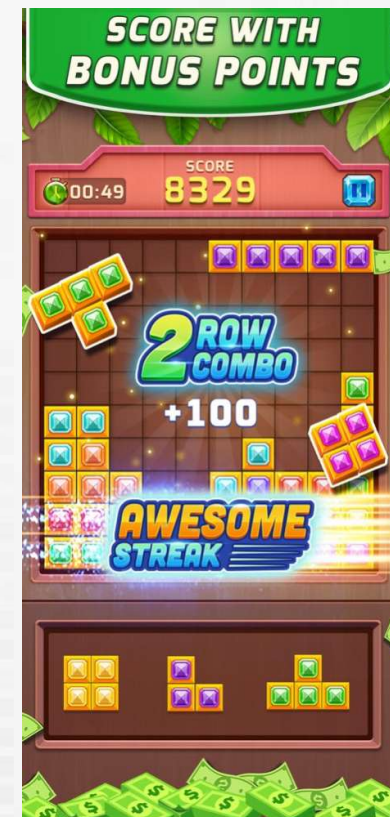


Audio & VFX

Audio System: Kết hợp AudioSource (nguồn phát) và AudioListener (đôi tai/camera) cho SFX & BGM.

Particle System: Hiệu ứng khói, lửa, nổ, lấp lánh khi ăn vật phẩm.

Hoạt ảnh, Âm thanh và Hiệu ứng hạt (Animation, Audio và Particle System)



| Event System, State & Coroutines

⚡ Event System & Game State

Event-Driven Architecture:

- Sử dụng C# Delegate/Action hoặc UnityEvent.
- Tách biệt rành mạch (Decoupling) giữa Lớp Logic (PlayerLogic) và Lớp View (PlayerView / UI).

Game State Manager:

- Quản lý vòng lặp trạng thái trò chơi: Main Menu → Playing → Pause → Win/GameOver.

🕒 Coroutines & Timers

Khái niệm Coroutine:

- Hàm cho phép tạm dừng thực thi và nhường quyền điều khiển cho Unity trong một khoảng thời gian (yield return).

Ứng dụng cốt lõi trong Gameplay:

- Bộ đếm giờ đếm ngược thời gian chơi (Countdown Timer).
- Hiệu ứng giật màn hình (Screen Shake), nhấp nháy khi bị thương.
- Trì hoãn thời gian sinh quái (Enemy Spawning delay).

Thực hành Mini Game



1. Phân tích Gameplay

Xác định Mechanic cốt lõi (ăn điểm, né chướng ngại vật), xây dựng sơ đồ Luồng dữ liệu (Data Flow) chuẩn 3 lớp.



2. Triển khai Cấu trúc

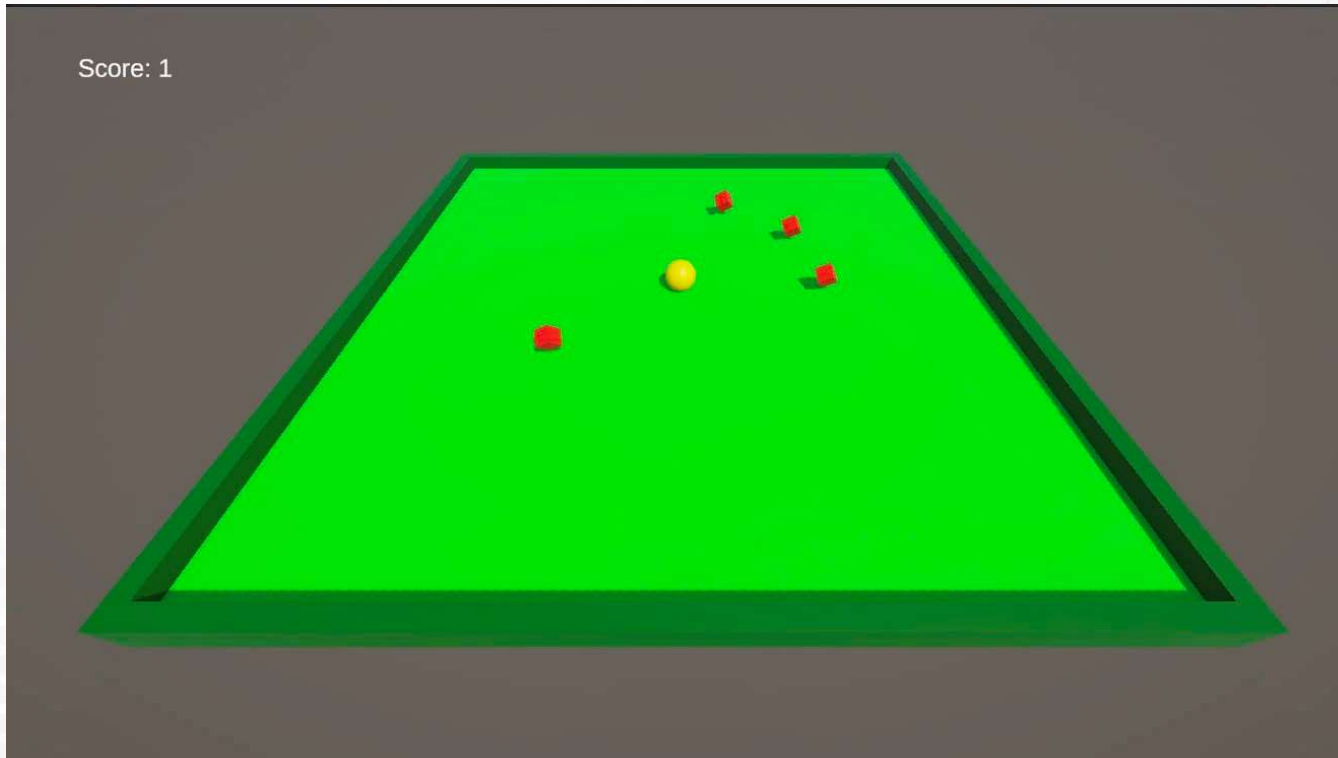
Tạo Scene mẫu (Roll-A-Ball / Platformer), gắn Rigidbody, thiết lập Trigger ăn Coin, kết nối UI TextMeshPro.



3. Hoàn thiện Game

Tích hợp âm thanh SFX, hiệu ứng Particle, xử lý Game State Win/Loss, đóng gói bản Build Executable (.exe).

Score: 1



Minigame

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG CDIT